Day: Date: / /

AUL Trees :- Assignment #4 (Aa gayi)

Properties :

① It is a BST

② |Height of left SubTree| − |H of R S·Tree| = {−1, 0, 1}

Modulus

Balance Factor → Calculate this for each node. Ans should be

* Duplicate elements Not allowed !!

* After every Insertion, check Balance factor of each node should be

Order of Insertion : x, y, z

(−1) − (−1) = 0

Calculate BF For Both Again

(−1) (0)

−1 − 1 = |−2| = ② (Wrong)

|−1| = 1

* Perform Rotation to Balance out.

In this case left.

Middle element Becomes Root

RR → left Rotation

x is unbalanced

* Balanced Now

* Another Case z, y, x

LL → Right Rotation

Right Rotation

|2|

|1|

|0|

z is Unbalanced

Yousaf

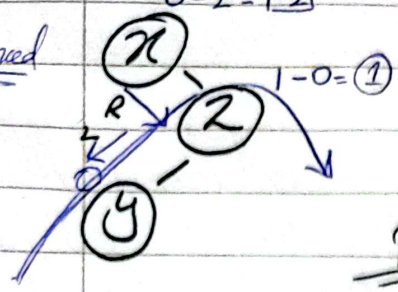
Day: 3rd

Date: 3,4 / 2 Rotations Required

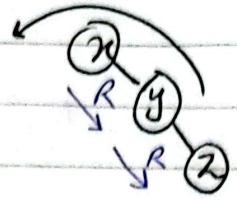
★ x, z, y

0 - 2 = -2

Unbalanced

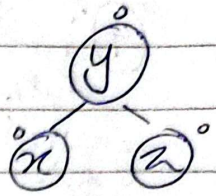


RL Rotation



First Right Rotation

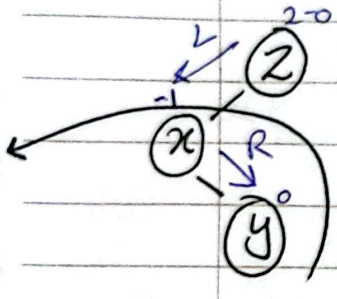
Left Rotation



4th

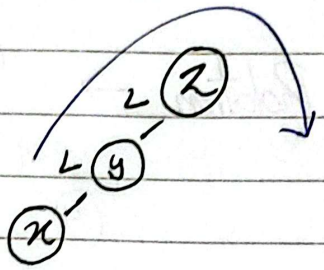
z, x, y

2 - 0 = 2

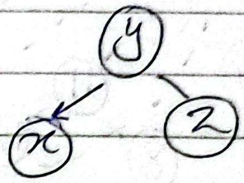


LR Rotation

left Rotation



Right Rotation



★ After Each Insertion check Balance factor. If unbalanced, ~~Check the last two insertions and~~ they will be amongst these 4.

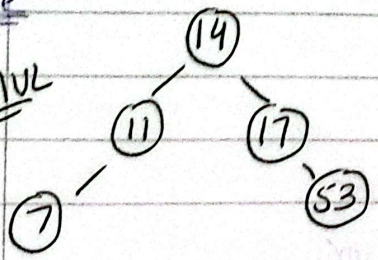
[RR, LL, RL, LR] then apply Required Case

★ Critical Node

Find this node and check its child. Child will be amongst the 4 Aforementioned Cases!

Unbalanced Node

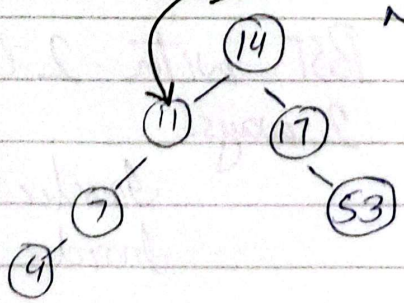
AVL



Insertion →

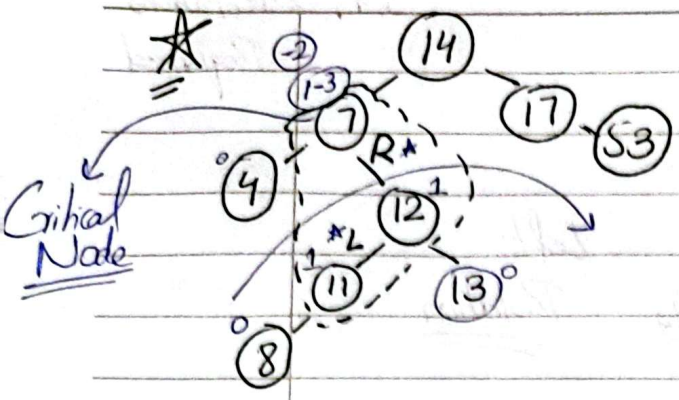
Critical Node

Not AVL



Yousaf

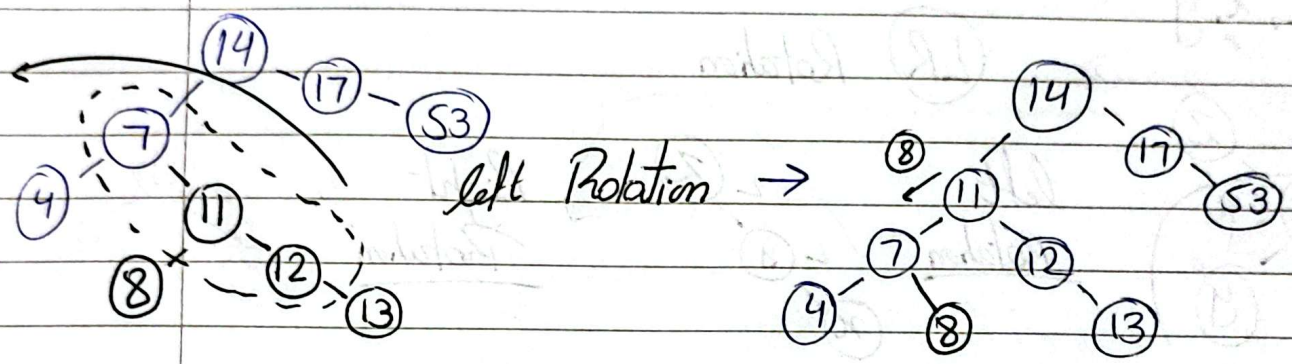
Day: Date: / /



After Identifying Critical Node find which type of Rotation is Needed!

* Since Became unbalanced After Inserting 8, we try and take 2 steps from 7 towards 8 and find its RL Rotation.

RL so Right then left Rotation!



Now how to place 8, it is to the left of (11) So its correct place is Right of (7)

* Deletion in AVL Tree!

* ① First Delete as you would do in BST.

② Now check BF from each node! Find CN if exists and Perform Required Rotation.

* Deletion in BST with 2 Childs.

2 ways:

Inorder Predecessor
Inorder Successor.

Yousaf

Day: Date: / /

Red Black Trees (BST)

Need:-

Frequent Operations

Balanced leads to $O(h) \rightarrow O(\log_2 n)$ *

BST
Avg $O(\log_2 n)$
Best $O(1)$
Worst $O(n)$

AVL → RB → Both Balanced
↳ *Self Balancing.

AVL ↔ RB Both $O(\log_2 n)$

So Why need?

RB

AVL → Many Rotations in Balancing After Insertion.

RB → Max 2 Rotations Required or Recoloring

AVL More Balanced
↳ Strictly Balanced
Searching Faster

RB
↳ Roughly height Balanced.
→ Insertion/Del faster.

Properties :-

* Self Balancing BST
* Node (Black or Red)
* Root Always Black
* Every leaf which is Nil is Black
* Node Red (Children Must Black)
* Every Path Node to Any of its NIL Descendants has same No of Black Nodes.

Yousaf

Day: Date: / /

Insertion:-

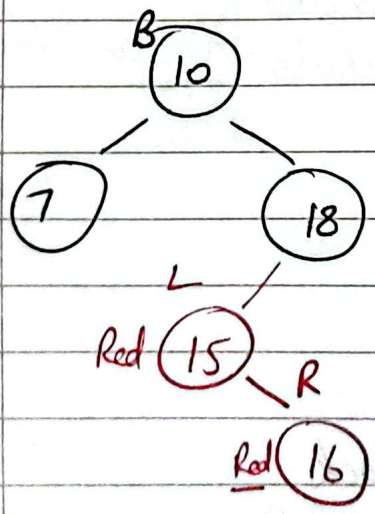
Insertion in AVL

↳ led to one of the 4 Cases [LL, RR, LR, RL]
Deletion in AVL may cause Many Rotations.

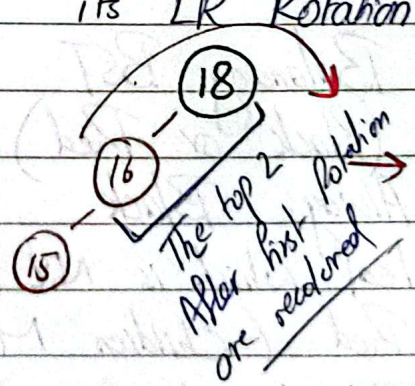
RB

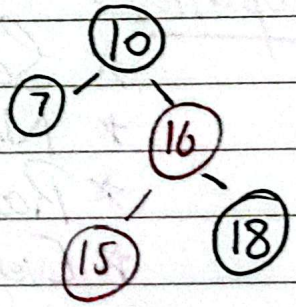
① Empty, Newnode Color: Black.
② !Empty, Newnode → leaf Color: Red
③ Newnode → Parent is Black Exit.
④ Parent of newNode (Red) then Color of Parent's Sibling (Uncle)

* If Red: Recolor & if parent's parent is not root then recolor & recheck.
* If Black or Null: Suitable rotation & recolor. & Recheck



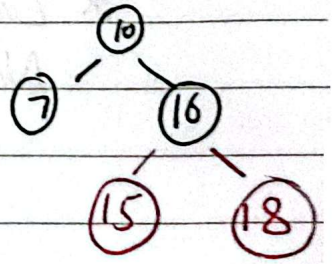
We inserted 16 and now Uncle is Black or NULL so we must Rotate and we can see its LR Rotation.





Now Recolor

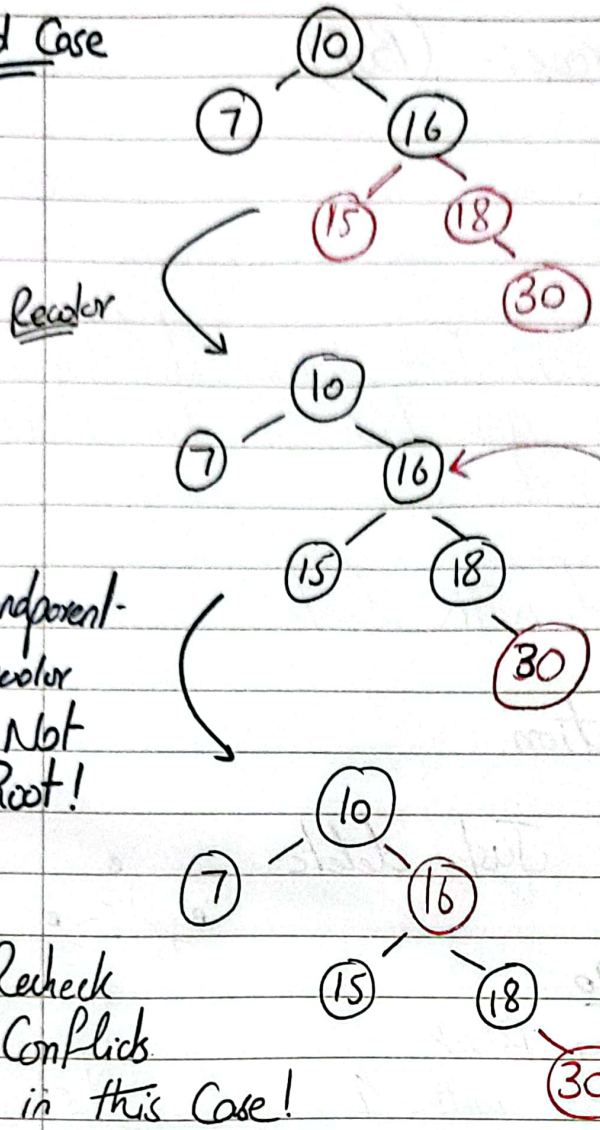
(In LR Case) 16, 18
New Added Biggest



Yousaf

Day: Date: / /

2nd Case



We add 30

* Uncle is Red only Recolor Uncle + Parent

* Parent's Parent Not Root then Recolor it.

* Now Recheck for Conflicts None in this Case!

Yousaf

Day: Date: / /

Deletion in Red Block Trees:- (BST)

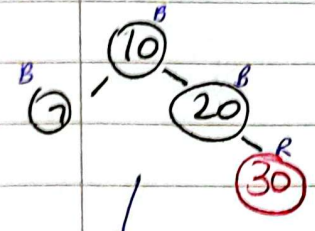
Red Node deletion easy.
Just Delete.

* **Important**:- In BST & RB Trees, When deleting a Node It's not like you delete it but you replace it.

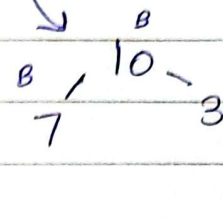
* Internal nodes replaced with leaf

1st- Standard BST deletion.

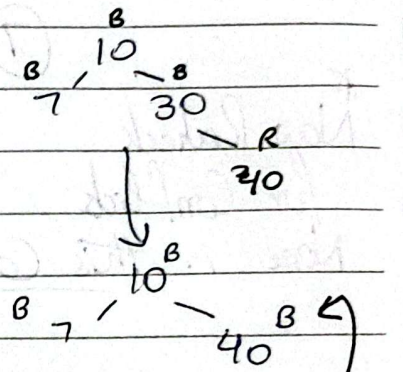
2nd Case-1 - Red Node Just delete



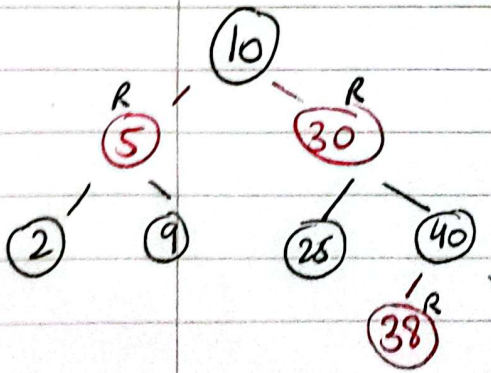
Delete 20
Deleting Black Node with 1 red child



Red so simply remove



We only Replace data value. It stayed black



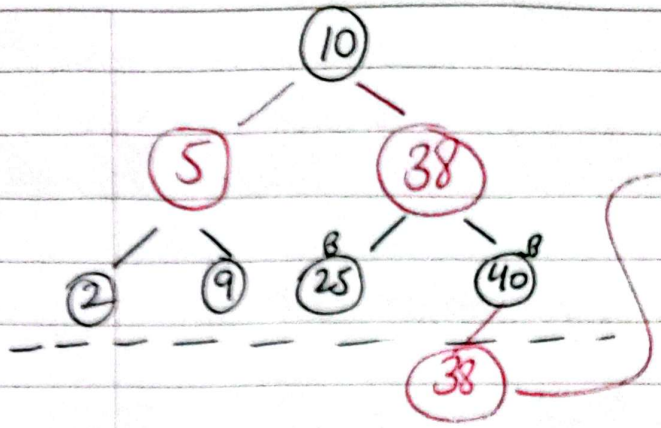
Delete 30
Deleting Red Node Internal

Go normally Replace With In Sucessor

Red Node leaf Complete delete

Yousaf

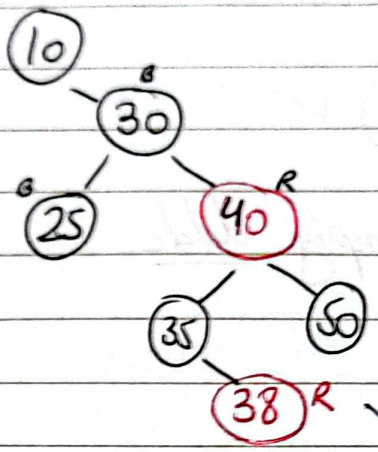
Day: Date: / /



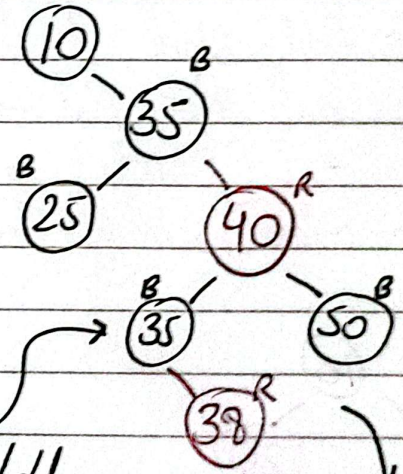
Red simply delete

Delete 30

deleting Black node with 1 red and 1 black child

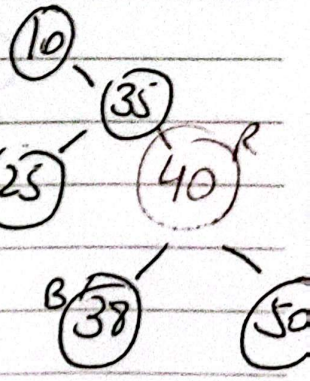


★ Replace with Inorder Successor!



★ Now call this delete recursively on Successor

So Block with 1 red child



Yousaf

Day: Date: / /

DB → double black

Step 1:- Normal BST deletion.

Step 2:- Case 1: If node to be deleted Red, Simple Remove

Case 2: If Root is DB, just remove DB

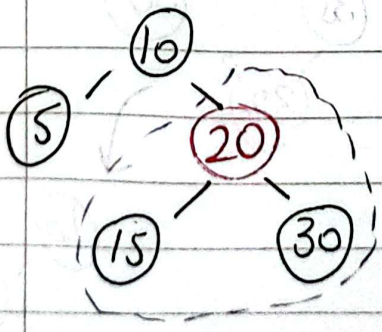
Case 3: If DB's Sibling is Black, & both children Black.

★ Internal Black, no problem (Replace) ✓

★ leaf Black, now problem. Can't Simply delete.

Case 3:-

→ Remove DB
→ Add Black to Parent
  ★ if P is Red → Becomes Black
  ★ if P Black → Becomes DB
→ Make Sibling Red.

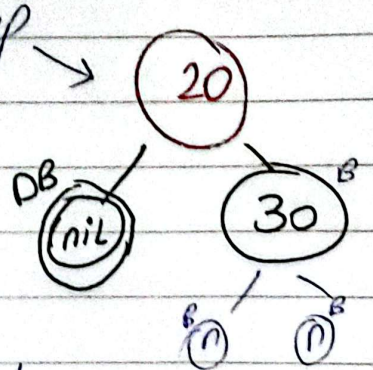


delete 15

Can't Simply delete Black leaf

→ Zoomed in View

Step →



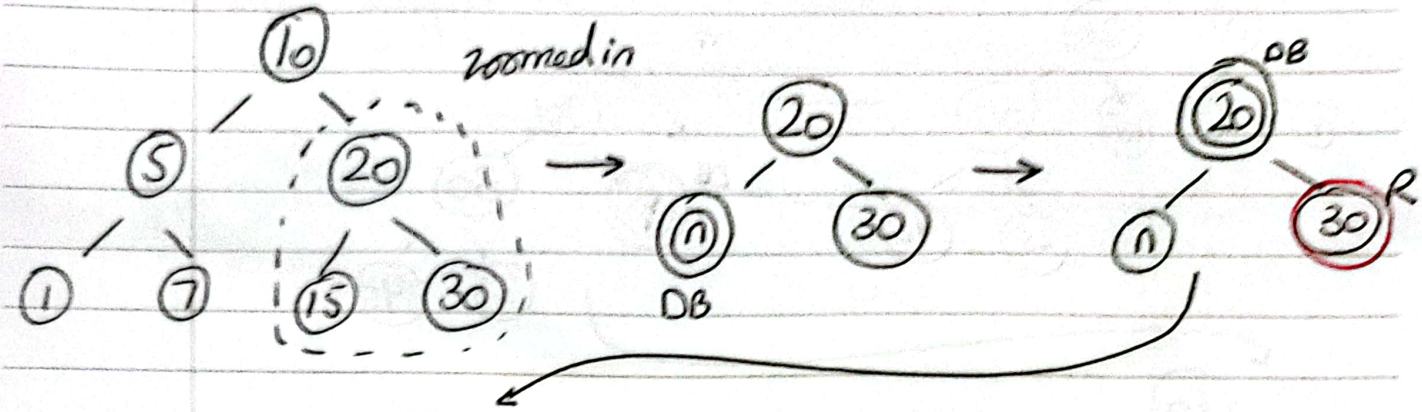
★ Since Child was NIL and it took place of parent So DB

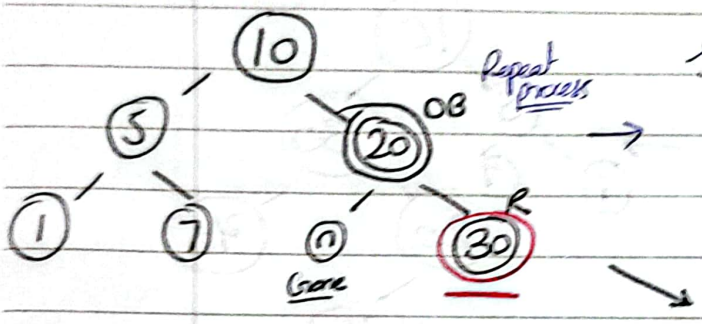
Yousaf

Day: Date: / /

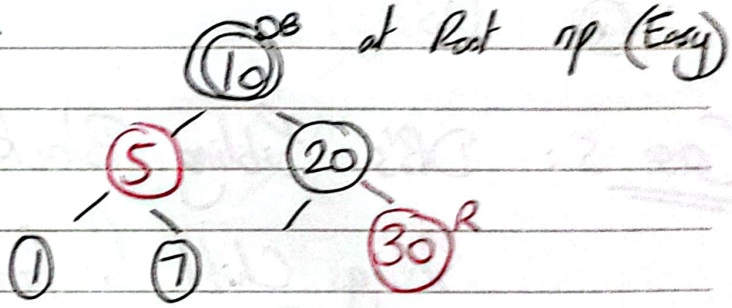
* Since Sibling & both child B So follow Steps (Easy)

Case of Parent Black





Since sibling B and 2 child Black
Give extra Black to parent.
Make sibling Red



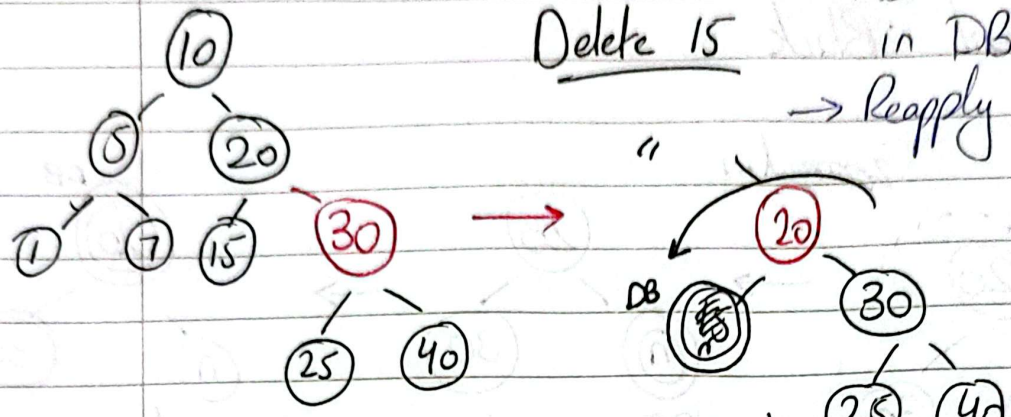
at Root np (Easy)

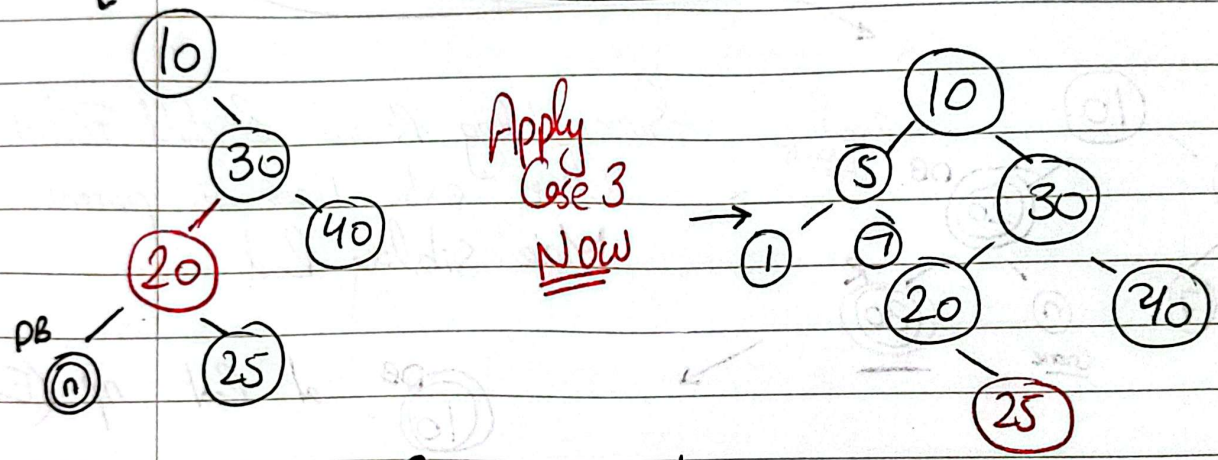
Yousaf

Day: Date: / /

Case 4: DB's Sibling Red:

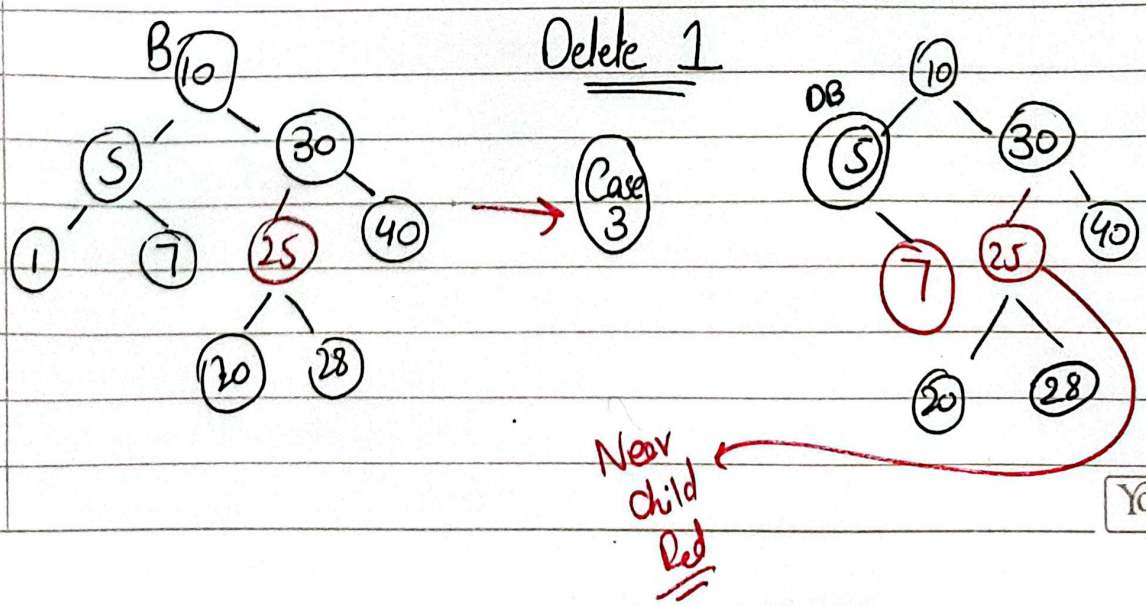
→ Swap Colors Pond Sibling
→ Rotate Parent of DB in DB direction
→ Reapply Cases.





Case 5: DB's Sibling Block

Sibling's Child far from DB is Black but close to DB is Red!



Yousaf

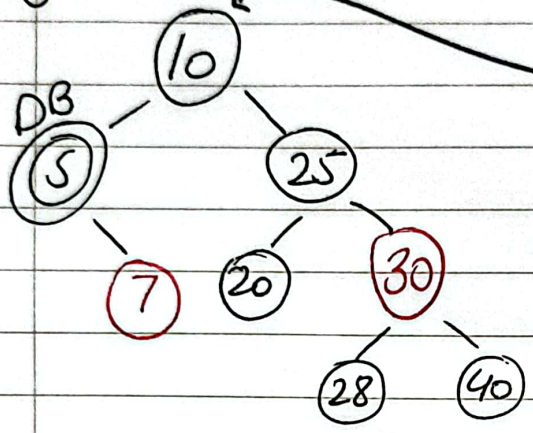
Day:

Date: / /

* Always Check for child first.
No need to check other if for red.

Case 5 Steps

→ Swap Color of DB's Sibling & Sibling's child near to DB.
→ Rotate Sibling in opposite direction of DB
→ Apply Case 6.



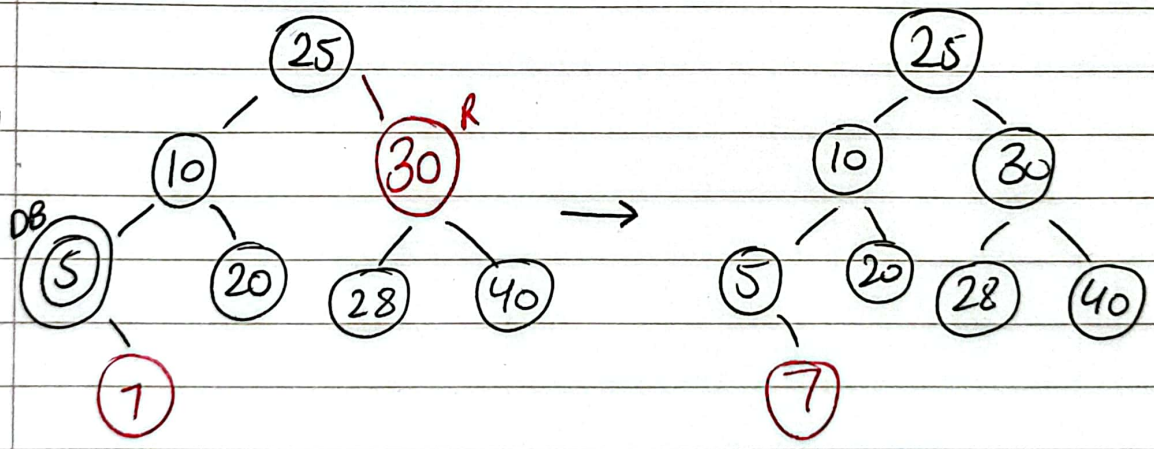
Steps DB's Sibling Black.
For Child Red.

(10) (25)

① Swap Color Parent & Sibling.
② Rotate P in DB direction
③ Remove DB.
④ Change Color Red child to Black.

① No swapping In this case but if Parent (10) was red. then both

After Rotation



Yousaf